

nanobot 实现框架与核心技术梳理

基于仓库源码的快速架构分析，重点关注 Agent 主循环、工具系统、通道层、会话层、WebUI/API 以及多 Agent 协作方式。

1. 一句话结论

nanobot 是一个以“消息总线 + 单主 Agent 核心 + 可插拔工具/通道/Provider”为中心的 AI Agent 框架。它不是传统意义上的大规模多 Agent 编排系统，但内置了明确的子 Agent 机制：主 Agent 可通过 spawn 工具创建后台 subagent，子 Agent 独立执行任务后把结果通过系统消息回灌主会话。

2. 总体架构

仓库的核心边界非常清晰：

- Channel 层负责接入外部平台，把消息转为 InboundMessage。
- MessageBus 负责把输入与输出解耦。
- AgentLoop 是主入口，负责会话、上下文、工具、记忆和状态机编排。
- AgentRunner 是通用 LLM 执行器，只负责“发模型 - 执行工具 - 继续迭代”。
- Provider 层封装不同模型后端。Tools / MCP / Skills 是能力扩展点。
- Session / memory / compact 层负责长期上下文与压缩。
- WebUI 与 OpenAI-compatible API 是不同入口，最终都复用同一套 AgentLoop。

核心数据流可以概括为：

```
Channel / API / WebSocket -> MessageBus -> AgentLoop -> ContextBuilder / SessionManager -> AgentRunner -> Tools / SubagentManager -> OutboundMessage -> Channel / WebUI
```

3. 核心技术栈

Python 3.11 + asyncio	主运行时，Agent Loop、消息总线、子任务、并发与超时控制都基于 asyncio。
Pydantic 配置	config/schema.py 定义显式配置模型，兼容 JSON/TOML 与 camelCase。
可插拔工具系统	ToolLoader 通过 pkgutil + entry_points 自动发现工具，ToolRegistry 统一注册与执行。
多 Provider 支持	OpenAI / Anthropic / Azure / Bedrock / OpenAI-compatible / Codex 等都抽象进 provider 层。
会话持久化与压缩	SessionManager、memory、Consolidator、AutoCompact 负责历史与长期记忆。
Runtime Events	RuntimeEventBus 让 WebUI 等订阅 turn / run / goal / model 状态变化。
MCP / 外部能力接入	支持 MCP 服务连接，扩展外部工具面。

4. nanobot 是如何搭建的

从代码结构看，nanobot 采用“核心稳定、边缘扩展”的搭建方式。AgentLoop 和 AgentRunner 是中心轴，周围挂着通道、工具、Provider、技能、MCP、会话和 WebUI。新增能力优先通过边缘层接入，而不是往主循环里硬塞逻辑。

关键实现特征：

- 消息入口统一：ChannelManager / API server 把输入汇入 MessageBus。
- 主循环统一：AgentLoop 处理路由、状态机、上下文构建、工具注册、持久化与输出。
- 执行器统一：AgentRunner 负责模型循环，支持工具调用、注入消息、超时和 hook。

- 工具统一：ToolLoader 扫描内置工具和插件，ToolRegistry 负责 schema、参数校验与执行。
- 会话统一：SessionManager / goal_state / autocompact 处理历史、目标和压缩。

5. 是否是多 Agent 系统

结论：是，但属于“主 Agent + 子 Agent”的轻量多 Agent，而不是一个复杂的多角色自治编队系统。主体决策仍由单个主 Agent 执行；多 Agent 能力主要体现在按需派生的后台 subagent。

证据链：

- tools/spawn.py 明确提供 spawn 工具，用于创建后台 subagent。
- agent/subagent.py 里有 SubagentManager、SubagentStatus 和后台 asyncio task 管理。
- 子 Agent 使用独立的 AgentRunner、独立工具注册表和独立 system prompt。
- 子 Agent 结束后会把结果包装成 system message，并带上 injected_event=subagent_result 回灌主会话。
- long_task.py 明确写明“没有 sub-agent orchestrator”，它只是主会话上的持续目标状态，不是多 Agent 编排。

6. 多 Agent 如何配合

- 1 用户在主会话发起任务。
- 2 主 Agent 通过普通推理决定是否需要并行处理，于是调用 spawn 工具。
- 3 SpawnTool 把当前会话上下文、消息 ID、工作区范围等信息传给 SubagentManager。
- 4 SubagentManager 为子 Agent 构建独立工具集、独立 prompt，并用 AgentRunner 在后台跑。
- 5 子 Agent 执行文件、shell、web、MCP 等工具，状态通过 hook 记录到 SubagentStatus。
- 6 子 Agent 完成后，通过 MessageBus 注入一条 system 消息回主会话。
- 7 AgentLoop 在处理 system 消息时把它识别为 subagent 结果，归入当前 session 的正常对话历史。
- 8 主 Agent 继续读到这条结果，再决定是否汇总、追问、继续分工或直接收束。

7. 设计上的重点

这套系统真正的技术重心，不在“造很多 Agent”，而在“把 Agent 做成可扩展的操作系统”。核心是：统一消息总线、统一会话语义、统一运行时事件、统一工具注册、统一 Provider 抽象，再在这个底座上按需挂子 Agent、MCP、技能和 WebUI。

换句话说，nanobot 的多 Agent 是“功能型增量”，不是“组织型中心”。它让主 Agent 可以把重活扔给后台子 Agent，但仍保持会话、记忆和最终解释权在主 Agent 这边。

8. 适合记住的三句话

1. nanobot = 单主 Agent 核心 + 可插拔边缘能力。
2. 它支持多 Agent，但主要通过 spawn 生成后台 subagent 来协作。
3. long_task 不是多 Agent，它只是主会话上的持续目标状态。

报告来源：nanobot/agent/loop.py、nanobot/agent/runner.py、nanobot/agent/subagent.py、nanobot/agent/tools/spawn.py、nanobot/agent/tools/long_task.py、nanobot/session/webui_turns.py、nanobot/api/server.py、nanobot/channels/manager.py、nanobot/bus/*。